

임베딩(Embedding)

원-핫 인코딩(One-hot encoding)

정의

예제

원-핫 인코딩(One-hot encoding)의 한계

워드 임베딩(Word Embedding)

1) 희소 표현(Sparse Representation)

2) 밀집 표현(Dense Representation)

3) 워드 임베딩(Word Embedding)

워드투벡터(Word2Vec)

1) 분산 표현(Distributed Representation)

2) CBOW(Continuous Bag of Words)

3) Skip-gram

4)네거티브 샘플링(Negative Sampling)

임베딩 벡터의 시각화(Embedding Visualization)

실습

자연어 처리에서 필수적으로 사용되는 단어의 표현 방법인 원-핫 인코딩(One-hot encoding)과 워드 임베딩(Word Embedding). 그리고 문서를 벡터화 하는 방법에 대해서 학습

원-핫 인코딩(One-hot encoding)

정의

- 원-핫 인코딩은 단어 집합의 크기를 벡터의 차원으로 하고, 표현하고 싶은 단어의 인덱스에 1의 값을 부여하고, 다른 인덱스에는 0을 부여하는 단어의 벡터 표현 방식
- 이렇게 표현된 벡터를 원-핫 벡터(One-hot vector)라고 함
- 원-핫 인코딩의 두 가지 과정
 - (1) 각 단어에 고유한 인덱스를 부여(정수 인코딩)
 - (2) 표현하고 싶은 단어의 인덱스의 위치에 1을 부여하고, 다른 단어의 인덱스의 위치에는 0을 부여

예제

문장 : 나는 딥러닝을 배운다

- 각 단어에 고유한 인덱스를 부여(정수 인코딩)

{ '나': 0, '는': 1, '답': 2, '러닝': 3, '을': 4, '배운다': 5 }

- 표현하고 싶은 단어의 인덱스의 위치에 1을 부여하고, 다른 단어의 인덱스의 위치에는 0을 부여

[1, 0, 0, 0, 0, 0] - 나
 [0, 1, 0, 0, 0, 0] - 는
 [0, 0, 1, 0, 0, 0] - 답
 [0, 0, 0, 1, 0, 0] - 러닝
 [0, 0, 0, 0, 1, 0] - 을
 [0, 0, 0, 0, 0, 1] - 배운다

원-핫 인코딩(One-hot encoding)의 한계

- 원-핫 인코딩은 단어의 개수가 늘어나면 벡터 차원도 함께 증가하여, 저장 공간이 비효율적으로 커지는 단점이 있음
- 단어 집합의 크기가 곧 벡터의 차원이 되어, 예를 들어 단어가 1,000개면 각각 1,000차원의 벡터로 표현됨
- 각 벡터는 하나의 값만 1이고 나머지는 모두 0이어 희소성이 심하며, 메모리 낭비가 큼
- 원-핫 벡터는 단어 간의 유사도를 표현할 수 없어, 예를 들어 '늑대'와 '강아지', '호랑이'와 '고양이'처럼 관련 있는 단어끼리도 구별하거나 연관성을 알 수 없음
- 유사성과 연관성이 필요한 검색 시스템에서는 효과적으로 연관 검색어나 확장 검색어를 처리할 수 없음

워드 임베딩(Word Embedding)

워드 임베딩(Word Embedding)은 단어를 벡터로 표현하는 것을 의미하며 단어를 밀집 표현으로 변환하는 방법임

1) 희소 표현(Sparse Representation)

- 벡터 또는 행렬(matrix)의 값이 대부분이 0으로 표현되는 방법
 - 원-핫 벡터는 희소 벡터(sparse vector)

- 희소 벡터의 문제점은 단어의 개수가 늘어나면 벡터의 차원이 한없이 커진다는 점과 공간 낭비
 - 단어 집합이 클수록 고차원의 벡터가 됨. 예를 들어 단어가 10,000개 있고 강아지란 단어의 인덱스는 5였다면 원 핫 벡터는 이렇게 표현되어야 했습니다.

Ex) 강아지 = [0 0 0 0 0 1 0 0 0 0 0 0 ... 종략 ... 0] # 이 때 1 뒤의 0의 수는 9994개.

2) 밀집 표현(Dense Representation)

- 밀집 표현은 벡터의 차원을 단어 집합의 크기로 상정하지 않고 사용자가 설정한 값으로 모든 단어의 벡터 표현의 차원을 맞추는 방법으로 0과 1만 가진 값이 아니라 실수값을 사용함

Ex) 강아지 = [0 0 0 0 1 0 0 0 0 0 0 0 ... 종략 ... 0] # 이 때 1 뒤의 0의 수는 9995개. 차원은 10,000

- 예를 들어 10,000개의 단어가 있을 때 강아지란 단어를 표현하기 위해서는 위와 같은 표현을 사용했음
- 밀집 표현을 사용하고, 사용자가 밀집 표현의 차원을 128로 설정한다면, 모든 단어의 벡터 표현의 차원은 128로 바뀌면서 모든 값이 실수가 됨

Ex) 강아지 = [0.2 1.8 1.1 -2.1 1.1 2.8 ... 종략 ...] # 이 벡터의 차원은 128

- 이 경우 벡터의 차원이 조밀해졌다고 하여 밀집 벡터(dense vector)라고 함

3) 워드 임베딩(Word Embedding)

- 단어를 밀집 벡터(dense vector)의 형태로 표현하는 방법
- 밀집 벡터를 워드 임베딩 과정을 통해 나온 결과라고 하여 **임베딩 벡터(embedding vector)**라고도 함
- 워드 임베딩 방법론으로는 LSA, Word2Vec, FastText, Glove 등이 있음
- 아래의 표는 앞서 배운 원-핫 벡터와 지금 배우고 있는 임베딩 벡터의 차이를 보여줌

-	원-핫 벡터	임베딩 벡터
차원	고차원(단어 집합의 크기)	저차원
다른 표현	희소 벡터의 일종	밀집 벡터의 일종
표현 방법	수동	훈련 데이터로부터 학습함
값의 타입	1과 0	실수

워드투벡터(Word2Vec)

- <https://arxiv.org/abs/1301.3781>
- 원-핫 벡터는 단어 간 유사도를 계산할 수 없다는 단점이 있음
- 단어 간 유사도를 반영할 수 있도록 단어의 의미를 벡터화 할 수 있는 방법
- 한국어 단어에 대해서 벡터 연산을 해볼 수 있는 사이트로 단어들(실제로는 Word2Vec 벡터)로 더하기, 빼기 연산을 할 수 있음
 - <https://word2vec.kr/search/>

고양이 + 애교 = 강아지
 한국 - 서울 + 도쿄 = 일본
 박찬호 - 야구 + 축구 = 호나우두

- 이런 연산이 가능한 이유는 각 단어 벡터가 단어 간 유사도를 반영한 값을 가지고 있기 때문

1) 분산 표현(Distributed Representation)

- 단어의 '의미'를 다차원 공간에 벡터화하는 방법
- 기본적으로 분포 가설(distributional hypothesis)이라는 가정 하에 만들어진 표현 방법
- 분포 가설은 '**비슷한 위치에서 등장하는 단어들은 비슷한 의미를 가진다**'라는 가정
 - 강아지란 단어는 귀엽다, 예쁘다, 애교 등의 단어가 주로 함께 등장하는데 분포 가설에 따라서 저런 내용을 가진 텍스트를 벡터화한다면 저 단어들은 의미적으로 가까운 단어가 됨

- 분산 표현은 분포 가설을 이용하여 단어들의 셋을 학습하고, 벡터에 단어의 의미를 여러 차원에 분산하여 표현함
- 원-핫 벡터 대비 벡터의 차원이 상대적으로 저차원
 - 예를 들어 단어가 10,000개 있고 인덱스가 1부터 시작한다고 하였을 때 강아지란 단어의 인덱스는 5였다면 강아지란 단어를 표현하는 원-핫 벡터는 다음과 같음

Ex) 강아지 = [0 0 0 0 1 0 0 0 0 0 0 0 ... 중략 ... 0]
1 뒤에 0이 9,995개가 있는 벡터

- Word2Vec로 임베딩 된 벡터는 굳이 벡터의 차원이 단어 집합의 크기가 될 필요가 없음
 - 강아지란 단어를 표현하기 위해 사용자가 설정한 차원을 가지는 벡터가 되면서 각 차원은 실수형의 값을 가짐

Ex) 강아지 = [0.2 0.3 0.5 0.7 0.2 ... 중략 ... 0.2]

- 요약하면 희소 표현이 고차원에 각 차원이 분리된 표현 방법이었다면, 분산 표현은 저차원에 **단어의 의미를 여러 차원에다가 분산하여 표현함**
- 이런 표현 방법을 사용하면 **단어 간 유사도**를 계산할 수 있음
- 이를 위한 학습 방법으로는 NNLM, RNNLM 등이 있으나 요즘에는 해당 방법들의 속도를 대폭 개선시킨 Word2Vec가 많이 쓰이고 있음

2) CBOW(Continuous Bag of Words)

- Word2Vec에는 CBOW(Continuous Bag of Words)와 Skip-Gram 두 가지 방식이 있음
- CBOW는 주변에 있는 단어들을 가지고, 중간에 있는 단어들을 예측하는 방법
- Skip-Gram은 중간에 있는 단어로 주변 단어들을 예측하는 방법
- 두 방법의 메커니즘 자체는 거의 동일함

예문 : "The fat cat sat on the mat"

- CBOW는 {"The", "fat", "cat", "on", "the", "mat"}으로부터 sat을 예측함
- 예측해야하는 단어 sat을 중심 단어(center word)라고 함
- 예측에 사용되는 단어들을 주변 단어(context word)라고 함

- 중심 단어를 예측하기 위해서 앞, 뒤로 몇 개의 단어를 볼지를 결정했다면 이 범위를 윈도우(window)라고 함
 - 예를 들어서 윈도우 크기가 2이고, 예측하고자 하는 중심 단어가 sat이라고 한다면 앞의 두 단어인 fat와 cat, 그리고 뒤의 두 단어인 on, the를 참고합니다. 윈도우 크기가 n이라고 한다면, 실제 중심 단어를 예측하기 위해 참고하려고 하는 주변 단어의 개수는 2n
 - 슬라이딩 윈도우(sliding window) : 윈도우 크기를 정했다면, 윈도우를 계속 움직여서 주변 단어와 중심 단어 선택을 바꿔가며 학습을 위한 데이터 셋을 만드는 방법

중심 단어 주변 단어

↓ ↓

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

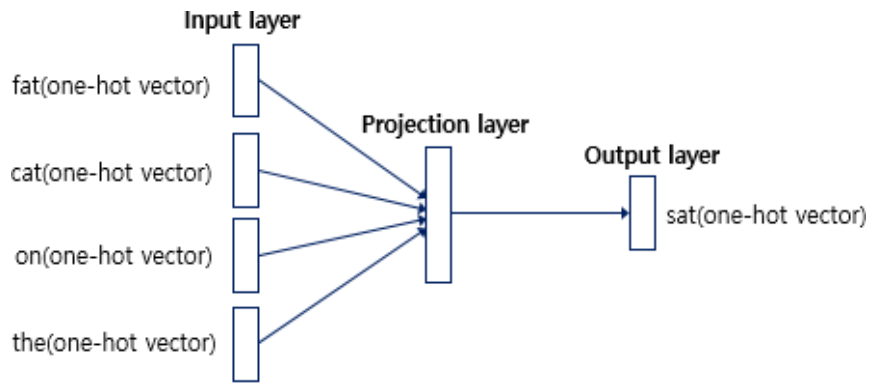
The fat cat sat on the mat

The fat cat sat on the mat

중심 단어	주변 단어
[1, 0, 0, 0, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0]
[0, 0, 1, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0]
[0, 0, 0, 1, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0]	[0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 1, 0]	[0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 0, 1]	[0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]

[그림 1] 윈도우 크기가 2일때, 슬라이딩 윈도우가 어떤 식으로 이루어지면서 데이터 셋을 만드는 방법

- Word2Vec에서 입력은 모두 원-핫 벡터가 되어야 하는데, [그림 1]은 중심 단어와 주변 단어를 어떻게 선택했을 때에 따라서 각각 어떤 원-핫 벡터가 되는지를 보여주며 또한 CBOW를 위한 전체 데이터 셋을 보여줌

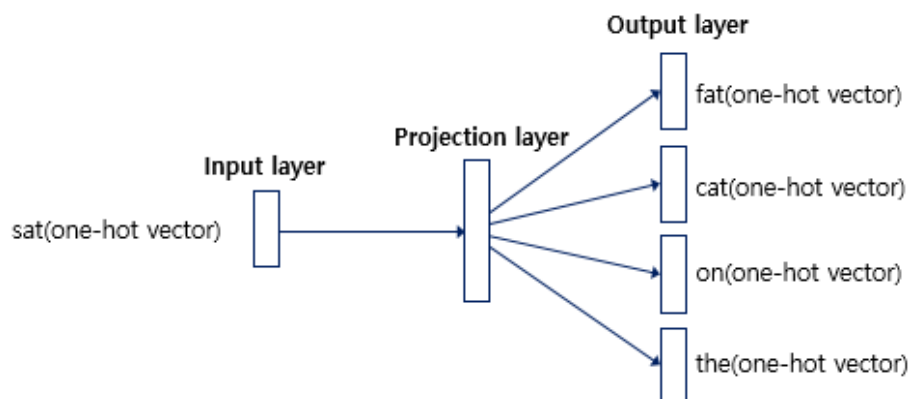


[그림 2] CBOW의 인공 신경망

- CBOW는 입력층(Input layer)의 입력으로서 앞, 뒤로 사용자가 정한 윈도우 크기 범위 안에 있는 주변 단어들의 원-핫 벡터가 들어가게 되고, 출력층(Output layer)에서 예측하고자 하는 중간 단어의 원-핫 벡터가 필요하며 Word2Vec의 학습을 위해서 이 중간 단어의 원-핫 벡터가 필요함
- Word2Vec은 딥 러닝 모델(Deep Learning Model)은 아님
 - Word2Vec은 입력층과 출력층 사이에 하나의 은닉층만이 존재함. 이렇게 은닉층(hidden Layer)이 1개인 경우에는 일반적으로 심층신경망(Deep Neural Network)이 아니라 얕은신경망(Shallow Neural Network)이라고 부름
- Word2Vec의 은닉층은 일반적인 은닉층과는 달리 활성화 함수가 존재하지 않으며 룩업 테이블이라는 연산을 담당하는 층으로 일반적인 은닉층과 구분하기 위해 투사층(projection layer)이라고 부르기도 함

3) Skip-gram

- Skip-gram은 CBOW를 이해했다면, 메커니즘 자체는 동일하기 때문에 쉽게 이해할 수 있음
- Skip-gram은 중심 단어에서 주변 단어를 예측함



[그림 3] Skip-gram 방식의 인공 신경망

- 중심 단어에 대해서 주변 단어를 예측하기 때문에, 투사층에서 벡터들의 평균을 구하는 과정은 없음
- 여러 논문에서 성능 비교를 진행했을 때, 전반적으로 Skip-gram이 CBOW보다 성능이 좋다고 알려져 있음

4)네거티브 샘플링(Negative Sampling)

- 대체적으로 Word2Vec를 사용한다고 하면 SGNS(Skip-Gram with Negative Sampling)을 사용함
- Skip-gram을 사용하는데, 네거티브 샘플링(Negative Sampling)이란 방법까지 추가로 사용함
- Word2Vec 모델에는 속도 문제가 있음
 - Word2Vec의 마지막 단계에서 출력층에 있는 소프트맥스 함수는 단어 집합 크기의 벡터 내의 모든 값을 0과 1사이의 값이면서 모두 더하면 1이 되도록 바꾸는 작업을 수행함. 그리고 이에 대한 오차를 구하고 모든 단어에 대한 임베딩을 조정함. 그 단어가 중심 단어나 주변 단어와 전혀 상관없는 단어라도 마찬가지로 수행. 만약 단어 집합의 크기가 수백만에 달한다면 이 작업은 굉장히 무거운 작업임
 - Word2Vec이 모든 단어 집합에 대해서 소프트맥스 함수를 수행하고, 역전파를 수행하므로 주변 단어와 상관 없는 모든 단어까지의 워드 임베딩 조정 작업을 수행한다는 겁니다. 만약 마지막 단계에서 '강아지'와 '고양이'와 같은 단어에 집중하고 있다면, Word2Vec은 사실 '돈가스'나 '컴퓨터'와 같은 연관 관계가 없는 수많은 단어의 임베딩을 조정할 필요가 없음
- 이를 조금 더 효율적으로 처리하기 위해 일부 단어 집합에 대해 이진 분류를 사용
 - 전체 단어 집합이 아니라 일부 단어 집합에 대해서만 고려
 - '강아지', '고양이', '애교'와 같은 주변 단어들을 가져오고 여기에 '돈가스', '컴퓨터', '회의실'과 같은 랜덤으로 선택된 주변 단어가 아닌 상관없는 단어들을 일부만 갖고옵니다. 이렇게 전체 단어 집합보다 훨씬 작은 단어 집합을 만들어 놓음
 - 마지막 단계를 이진 분류 문제로 수행
 - Word2Vec은 주변 단어들을 긍정(positive)으로 두고 랜덤으로 샘플링 된 단어들을 부정(negative)으로 둔 다음에 이진 분류 문제를 수행함
 - 이는 기존의 다중 클래스 분류 문제를 이진 분류 문제로 바꾸면서도 연산량에 있어서 훨씬 효율적임

임베딩 벡터의 시각화(Embedding Visualization)

- <https://projector.tensorflow.org/>

실습

- https://github.com/cserock/colab-examples/blob/main/05_One_hot_encoding_Word_Embedding.ipynb